

Chapter 12 — Heap / Priority Queue

0. What You Will Learn in This Chapter

Heap / Priority Queue is one of the most important patterns for problems involving repeated access to the **smallest**, **largest**, or **highest-priority** item.

A heap is useful when you do not need everything sorted, but you repeatedly need the current best item.

By the end of this chapter, you should understand:

- What a heap is
- What a priority queue is
- Min heap vs max heap
- How C++ `priority_queue` works
- When to use heap instead of sorting
- When to use heap instead of Quickselect
- Top K problems
- K-way merge problems
- Scheduling problems
- Streaming median
- Two-heap pattern
- Heap with custom comparator
- Lazy deletion idea
- Common heap mistakes

Core LeetCode problems covered in this chapter:

- 1.1. Kth Largest Element in a Stream
 - 2.2. Kth Largest Element in an Array
 - 3.3. Top K Frequent Elements
 - 4.4. Top K Frequent Words
 - 5.5. K Closest Points to Origin
 - 6.6. Last Stone Weight
 - 7.7. Merge k Sorted Lists
 - 8.8. Kth Smallest Element in a Sorted Matrix
 - 9.9. Find Median from Data Stream
 - 10.10. Meeting Rooms II
 - 11.11. Task Scheduler
 - 12.12. Reorganize String
 - 13.13. Distant Barcodes
 - 14.14. Furthest Building You Can Reach
 - 15.15. IPO
 - 16.16. Smallest Range Covering Elements from K Lists
 - 17.17. Sliding Window Median, advanced lazy deletion
-

1. What Is a Heap?

A heap is a data structure that lets you quickly access the minimum or maximum element.

There are two main types:

1.1 Max Heap

A max heap gives quick access to the largest element.

In C++, default `priority_queue<int>` is a max heap.

```
priority_queue<int> pq;
```

```
pq.push(5);
```

```
pq.push(1);
```

```
pq.push(10);
```

```
cout << pq.top(); // 10
```

1.2 Min Heap

A min heap gives quick access to the smallest element.

In C++, use:

```
priority_queue<int, vector<int>, greater<int>>> pq;
```

Example:

```
priority_queue<int, vector<int>, greater<int>>> pq;
```

```
pq.push(5);
```

```
pq.push(1);
```

```
pq.push(10);
```

```
cout << pq.top(); // 1
```

2. Why Heap Exists

Sorting gives complete order.

Example:

```
nums = {5, 1, 9, 3, 7}
```

Sorting:

```
{1, 3, 5, 7, 9}
```

But many problems do not need full order.

They only need:

- current largest
- current smallest
- top k elements
- kth largest so far
- next smallest among many sorted lists
- earliest ending meeting
- most frequent remaining character

A heap avoids fully sorting when you only need repeated best-item access.

3. Core Operations

For a heap with size n :

push	$O(\log n)$
pop	$O(\log n)$
top	$O(1)$
size	$O(1)$
empty	$O(1)$

4. Heap vs Sorting vs Quickselect

Sorting	Need full sorted order	$O(n \log n)$
Heap	Repeated min/max or streaming	$O(n \log k)$ often
Quickselect	One-time kth/top k unordered	$O(n)$ average
Bucket Sort	Frequency counts with bounded count	$O(n)$

4.1 When Heap Beats Sorting

If you need top k and k is small:

Heap: $O(n \log k)$

Sort: $O(n \log n)$

Example:

Find k closest points where $k \ll n$

4.2 When Sorting Is Simpler

If output must be sorted, sorting may be easier.

Example:

Return all intervals sorted by start.

Heap is unnecessary.

4.3 When Quickselect Beats Heap

If input is fixed and you only need one k th element:

Quickselect average $O(n)$

Heap $O(n \log k)$

Sort $O(n \log n)$

But heap is better for streaming.

5. When to Use Heap

Use a heap when the problem says or implies:

5.1 Top K / Kth

Clues:

- kth largest
- kth smallest
- top k frequent
- k closest
- maintain kth largest in stream

Examples:

- Kth Largest Element in a Stream
 - Top K Frequent Elements
 - K Closest Points
-

5.2 Repeatedly Take Largest or Smallest

Clues:

- repeatedly choose largest
- repeatedly choose smallest
- simulate priority
- smash two largest
- assign highest priority item

Examples:

- Last Stone Weight
 - Reorganize String
 - Task Scheduler
-

5.3 Merge Sorted Sources

Clues:

- k sorted lists
- k sorted arrays
- smallest range covering lists
- kth smallest in sorted matrix

Examples:

- Merge k Sorted Lists
 - Smallest Range Covering Elements from K Lists
-

5.4 Scheduling / Intervals

Clues:

- meeting rooms
- earliest ending

- minimum number of resources
- process tasks by time

Examples:

- Meeting Rooms II
 - Single-Threaded CPU
 - Employee Free Time
-

5.5 Streaming Data

Clues:

- data arrives one at a time
- maintain median
- maintain kth largest
- online queries

Examples:

- Find Median from Data Stream
 - Kth Largest Element in a Stream
-

6. When NOT to Use Heap

6.1 Need Full Sorted Order Once

Use sorting.

Heap can sort by repeatedly popping, but that is usually not simpler.

6.2 Need Fast Search for Arbitrary Value

Heap only gives fast access to top.

It does not support fast arbitrary lookup.

Use hash map/set.

6.3 Need Remove Arbitrary Element Efficiently

C++ priority_queue does not support removing arbitrary values.

For that, use:

- multiset
 - heap + lazy deletion
 - indexed heap, advanced
 - balanced tree
-

6.4 Need Range Queries

Heap is not for range sum/min/max queries across arbitrary ranges.

Use:

- prefix sum
 - segment tree
 - Fenwick tree
 - sparse table
 - monotonic deque
-

7. C++ Priority Queue Basics

7.1 Max Heap

```
priority_queue<int> pq;
```

Largest item is on top.

7.2 Min Heap

```
priority_queue<int, vector<int>, greater<int>> pq;
```

Smallest item is on top.

7.3 Common Operations

```
pq.push(x);  
pq.top();  
pq.pop();  
pq.size();  
pq.empty();
```

Important:

```
pq.pop();
```

removes top but does not return it.

Correct:

```
int x = pq.top();  
pq.pop();
```

7.4 Heap of Pairs

Default max heap for pairs:

```
priority_queue<pair<int, int>> pq;
```

Pairs compare lexicographically:

2. first
3. second if first ties

Min heap of pairs:

```
priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
```

8. Custom Comparator for Heap

C++ priority_queue comparator feels reversed compared to sort.

For priority_queue, comparator returns true if a has **lower priority** than b.

8.1 Custom Min Heap for Vector

Example: sort by first value ascending.

```
struct Compare {
    bool operator()(const vector<int>& a, const vector<int>& b) {
        return a[0] > b[0];
    }
};
```

Use:

```
priority_queue<vector<int>, vector<vector<int>>, Compare> pq;
```

Why >?

Because if $a[0] > b[0]$, then a should come after b.

So smaller first value rises to top.

8.2 Lambda Comparator

```
auto cmp = [](const vector<int>& a, const vector<int>& b) {
    return a[0] > b[0];
};

priority_queue<vector<int>, vector<vector<int>>, decltype(cmp)> pq(cmp);
```

9. Core Heap Templates

Template 1 — Max Heap

```
priority_queue<int> pq;

for (int x : nums) {
    pq.push(x);
}

int largest = pq.top();
```

Template 2 — Min Heap

```
priority_queue<int, vector<int>, greater<int>> pq;

for (int x : nums) {
    pq.push(x);
}

int smallest = pq.top();
```

Template 3 — Top K Largest Using Min Heap

Keep heap size at most k.

```
priority_queue<int, vector<int>, greater<int>> pq;

for (int x : nums) {
    pq.push(x);

    if (pq.size() > k) {
        pq.pop();
    }
}

return pq.top(); // kth largest
```

Why min heap?

The heap stores the k largest seen so far.

The smallest among those k is the kth largest.

Template 4 — Top K Smallest Using Max Heap

```
priority_queue<int> pq;

for (int x : nums) {
    pq.push(x);

    if (pq.size() > k) {
        pq.pop();
    }
}
```

Heap stores k smallest.

Top is largest among the k smallest.

Template 5 — K-Way Merge

```
using T = tuple<int, int, int>; // value, listIndex, elementIndex

priority_queue<T, vector<T>, greater<T>> pq;

for (int i = 0; i < lists.size(); i++) {
    pq.push({lists[i][0], i, 0});
}

while (!pq.empty()) {
    auto [value, listIndex, elementIndex] = pq.top();
```

```

pq.pop();

// process value

if (elementIndex + 1 < lists[listIndex].size()) {
    pq.push({lists[listIndex][elementIndex + 1], listIndex, elementIndex + 1});
}
}

```

Template 6 — Two Heaps for Median

```

priority_queue<int> small; // max heap
priority_queue<int, vector<int>, greater<int>> large; // min heap

```

Invariant:

```

small contains lower half
large contains upper half
small.size() == large.size() or small.size() == large.size() + 1

```

Template 7 — Meeting Rooms Min Heap

```

sort(intervals.begin(), intervals.end());

priority_queue<int, vector<int>, greater<int>> endTimes;

for (auto& interval : intervals) {
    int start = interval[0];
    int end = interval[1];

    if (!endTimes.empty() && endTimes.top() <= start) {
        endTimes.pop();
    }

    endTimes.push(end);
}

return endTimes.size();

```

10. Major Heap Variations

Kth largest	min heap size k	Kth Largest Stream

Variation	Heap Type	Example
Kth smallest	max heap size k	kth smallest
Top K frequent	min heap by frequency	Top K Frequent
K closest	max heap by distance size k	K Closest Points
Repeated largest	max heap	Last Stone Weight
K-way merge	min heap	Merge k Lists
Scheduling	min heap of end times	Meeting Rooms II
Median stream	two heaps	Median Finder
Greedy resource use	min/max heap	Furthest Building
Available projects	max heap profit	IPO

11. Full Problem Walkthrough 1 — LeetCode 703: Kth Largest Element in a Stream

11.1 Problem Summary

Design a class that receives numbers one by one and returns kth largest after each insertion.

Example:

k = 3
nums = {4, 5, 8, 2}

After adding values, return current kth largest.

11.2 How to Identify Heap

Problem says:

- stream
- kth largest after each insertion
- dynamic data

Quickselect does not fit streaming.

Use min heap of size k.

11.3 Intuition

Keep only the k largest elements seen so far.

The smallest among these k elements is the kth largest.

Use min heap.

11.4 C++ Code

```
class KthLargest {
private:
    int k;
    priority_queue<int, vector<int>, greater<int>> pq;

public:
    KthLargest(int k, vector<int>& nums) {
        this->k = k;

        for (int x : nums) {
            add(x);
        }

        int add(int val) {
            pq.push(val);

            if (pq.size() > k) {
                pq.pop();
            }

            return pq.top();
        }
    };
};
```

11.5 Dry Run

k = 3
nums = {4,5,8,2}

Heap keeps 3 largest:

{4,5,8}

Top is 4.

Add 3:

top remains 4

Add 10:

Heap becomes:

{5,8,10}

Top is 5.

11.6 Complexity

Constructor:

$O(n \log k)$

Each add:

$O(\log k)$

Space:

$O(k)$

11.7 Common Mistakes

- Using max heap
- Keeping all elements
- Returning largest instead of kth largest
- Popping when size equals k instead of greater than k

12. Full Problem Walkthrough 2 — LeetCode 215: Kth Largest Element in an Array

12.1 Problem Summary

Return kth largest element in an unsorted array.

Example:

nums = {3,2,1,5,6,4}
k = 2

Output:

5

12.2 Heap Approach

Use min heap of size k.

12.3 C++ Code

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<int>> pq;

        for (int x : nums) {
            pq.push(x);

            if (pq.size() > k) {
```

```

        pq.pop();
    }
}

return pq.top();
}
};

```

12.4 Complexity

Time:

$O(n \log k)$

Space:

$O(k)$

12.5 Comparison

Sort	$O(n \log n)$	$O(1) / O(\log n)$
Heap	$O(n \log k)$	$O(k)$
Quickselect	$O(n)$ average	$O(1)$

Heap is simple and predictable.

13. Full Problem Walkthrough 3 — LeetCode 347: Top K Frequent Elements

13.1 Problem Summary

Given array `nums`, return `k` most frequent elements.

Example:

```

nums = {1,1,1,2,2,3}
k = 2

```

Output:

```
{1,2}
```

13.2 How to Identify Heap

Need top `k` by frequency.

Use frequency map, then min heap by frequency of size `k`.

13.3 C++ Code

```
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        unordered_map<int, int> freq;

        for (int x : nums) {
            freq[x]++;
        }

        priority_queue<pair<int, int>,
                      vector<pair<int, int>>,
                      greater<pair<int, int>>> pq;

        for (auto& p : freq) {
            int value = p.first;
            int count = p.second;

            pq.push({count, value});

            if (pq.size() > k) {
                pq.pop();
            }
        }

        vector<int> result;

        while (!pq.empty()) {
            result.push_back(pq.top().second);
            pq.pop();
        }

        return result;
    }
};
```

13.4 Explanation

Heap stores:

{frequency, value}

Min heap keeps smallest frequency on top.

If heap size exceeds k , remove lowest frequency.
At end, heap contains k most frequent values.

13.5 Complexity

Let u be unique values.

Time:

$O(n + u \log k)$

Space:

$O(u + k)$

13.6 Alternatives

- bucket sort: $O(n)$
 - Quickselect: average $O(n)$
 - sorting: $O(u \log u)$
-

14. Full Problem Walkthrough 4 — LeetCode 692: Top K Frequent Words

14.1 Problem Summary

Return k most frequent words.

Tie-breaker:

- higher frequency first
- if tie, lexicographically smaller word first

Example:

`words = {"i","love","leetcode","i","love","coding"}`
`k = 2`

Output:

`{"i","love"}`

14.2 Heap Comparator Challenge

For top k , we use a min heap of the “worst” among selected k .

Worst means:

- lower frequency is worse
- if same frequency, lexicographically larger word is worse

So the heap top should be the word to remove first.

14.3 C++ Code

```
class Solution {  
public:
```

```

struct Compare {
    bool operator()(const pair<string, int>& a, const pair<string, int>& b) {
        if (a.second != b.second) {
            return a.second > b.second;
        }

        return a.first < b.first;
    }
};

vector<string> topKFrequent(vector<string>& words, int k) {
    unordered_map<string, int> freq;

    for (string& word : words) {
        freq[word]++;
    }

    priority_queue<pair<string, int>,
        vector<pair<string, int>>,
        Compare> pq;

    for (auto& p : freq) {
        pq.push({p.first, p.second});

        if (pq.size() > k) {
            pq.pop();
        }
    }

    vector<string> result;

    while (!pq.empty()) {
        result.push_back(pq.top().first);
        pq.pop();
    }

    reverse(result.begin(), result.end());

    return result;
}
};

```

14.4 Why Reverse?

The heap pops worst-to-best among the kept top k.
Final required output is best-to-worst.
So reverse.

14.5 Comparator Explanation

```
if (a.second != b.second) {  
    return a.second > b.second;  
}
```

Lower frequency should have higher priority for removal.
For same frequency:

```
return a.first < b.first;
```

Lexicographically larger word is worse and should be closer to top.
This comparator is tricky; sorting version is often simpler.

14.6 Complexity

Let u be unique words.

Time:

```
O(n + u log k)
```

Space:

```
O(u + k)
```

15. Full Problem Walkthrough 5 — LeetCode 973: K Closest Points to Origin

15.1 Problem Summary

Return k closest points to origin.

15.2 Heap Approach

Use max heap of size k by distance.

Why max heap?

We keep k closest points.

The farthest among those k is on top.

If a closer point arrives, remove the farthest.

15.3 C++ Code

```
class Solution {  
public:
```

```

int dist(vector<int>& p) {
    return p[0] * p[0] + p[1] * p[1];
}

vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
    priority_queue<pair<int, int>> pq;

    for (int i = 0; i < points.size(); i++) {
        pq.push({dist(points[i]), i});

        if (pq.size() > k) {
            pq.pop();
        }
    }

    vector<vector<int>> result;

    while (!pq.empty()) {
        result.push_back(points[pq.top().second]);
        pq.pop();
    }

    return result;
}
};

```

15.4 Complexity

Time:

$O(n \log k)$

Space:

$O(k)$

15.5 Alternatives

- sort by distance: $O(n \log n)$
- Quickselect: average $O(n)$

16. Full Problem Walkthrough 6 — LeetCode 1046: Last Stone Weight

16.1 Problem Summary

Each turn, smash the two heaviest stones.
If equal, both destroyed.
If different, new stone has weight difference.
Return final stone weight or 0.
Example:

stones = {2,7,4,1,8,1}

Output:

1

16.2 How to Identify Heap

Need repeatedly access two largest values.
Use max heap.

16.3 C++ Code

```
class Solution {
public:
    int lastStoneWeight(vector<int>& stones) {
        priority_queue<int> pq;

        for (int stone : stones) {
            pq.push(stone);
        }

        while (pq.size() > 1) {
            int a = pq.top();
            pq.pop();

            int b = pq.top();
            pq.pop();

            if (a != b) {
                pq.push(a - b);
            }
        }

        if (pq.empty()) {
            return 0;
        }

        return pq.top();
    }
};
```

16.4 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

16.5 Common Mistake

Using sorting repeatedly.

That can become inefficient.

Heap naturally supports repeated largest extraction.

17. Full Problem Walkthrough 7 — LeetCode 23: Merge k Sorted Lists

17.1 Problem Summary

Given k sorted linked lists, merge them into one sorted linked list.

17.2 How to Identify Heap

Need repeatedly choose the smallest current node among k lists.

Use min heap.

17.3 C++ Code

```
class Solution {
public:
    struct Compare {
        bool operator()(ListNode* a, ListNode* b) {
            return a->val > b->val;
        }
    };

    ListNode* mergeKLists(vector<ListNode*>& lists) {
        priority_queue<ListNode*, vector<ListNode*>, Compare> pq;

        for (ListNode* node : lists) {
            if (node != nullptr) {
                pq.push(node);
            }
        }

        ListNode dummy(0);
```

```

ListNode* tail = &dummy;

while (!pq.empty()) {
    ListNode* node = pq.top();
    pq.pop();

    tail->next = node;
    tail = tail->next;

    if (node->next != nullptr) {
        pq.push(node->next);
    }
}

return dummy.next;
}
};

```

17.4 Complexity

Let:

- N = total number of nodes
- k = number of lists

Time:

$O(N \log k)$

Space:

$O(k)$

17.5 Common Mistakes

- pushing null pointers
- comparing node pointers instead of values
- forgetting to push next node
- not using dummy node

18. Full Problem Walkthrough 8 — LeetCode 295: Find Median from Data Stream

18.1 Problem Summary

Design a data structure supporting:

```
addNum(num)
findMedian()
```

Numbers arrive one at a time.

18.2 How to Identify Two Heaps

Need median dynamically.

Median depends on lower half and upper half.

Use:

- max heap for lower half
 - min heap for upper half
-

18.3 Invariants

small max heap contains lower half

large min heap contains upper half

$\text{small.size()} \geq \text{large.size()}$

$\text{small.size()} - \text{large.size()} \leq 1$

Also:

all values in small $\leq$ all values in large

18.4 C++ Code

```
class MedianFinder {
private:
    priority_queue<int> small;
    priority_queue<int, vector<int>, greater<int>> large;

public:
    MedianFinder() {
    }

    void addNum(int num) {
        small.push(num);

        large.push(small.top());
        small.pop();

        if (large.size() > small.size()) {
            small.push(large.top());
            large.pop();
        }
    }
}
```

```
double findMedian() {
    if (small.size() > large.size()) {
        return small.top();
    }

    return ((double)small.top() + large.top()) / 2.0;
}
};
```

18.5 Explanation

When adding:

4. Push to small.
5. Move largest from small to large.
6. If large becomes bigger, move smallest from large back to small.

This maintains order and size balance.

18.6 Complexity

Each insertion:

$O(\log n)$

Median query:

$O(1)$

Space:

$O(n)$

18.7 Common Mistakes

- Not balancing sizes
 - Using integer division for median
 - Allowing upper half to be bigger
 - Forgetting all lower half values must be $\leq$ upper half values
-

19. Full Problem Walkthrough 9 — LeetCode 253: Meeting Rooms II

19.1 Problem Summary

Given meeting intervals, return minimum number of rooms required.

19.2 How Heap Helps

Sort meetings by start time.

Use min heap of end times.

Heap top is earliest room that becomes free.

19.3 C++ Code

```
int minMeetingRooms(vector<vector<int>>& intervals) {
    if (intervals.empty()) {
        return 0;
    }

    sort(intervals.begin(), intervals.end());

    priority_queue<int, vector<int>, greater<int>> endTimes;

    for (auto& interval : intervals) {
        int start = interval[0];
        int end = interval[1];

        if (!endTimes.empty() && endTimes.top() <= start) {
            endTimes.pop();
        }

        endTimes.push(end);
    }

    return endTimes.size();
}
```

19.4 Why It Works

Each active meeting occupies one room.

When earliest ending meeting ends before current starts, reuse that room.

Heap size is number of rooms currently needed.

Maximum heap size during scan is answer.

In this specific implementation, final heap size also equals the max needed because we pop one available room before pushing each meeting. A version tracking `maxSize` is also fine.

19.5 Complexity

Time:

$O(n \log n)$

Space:

$O(n)$

20. Full Problem Walkthrough 10 — LeetCode 1642: Furthest Building You Can Reach

20.1 Problem Summary

You have heights, bricks, and ladders.

Move from building i to $i+1$.

If next is higher, climb difference.

You can use bricks equal to difference or one ladder.

Return furthest building index reachable.

20.2 How to Identify Heap

Need decide where to use ladders.

Greedy idea:

Use ladders on largest climbs.

Use bricks on smaller climbs.

As you scan, keep climbs assigned to ladders in a min heap.

If number of ladder climbs exceeds ladders, convert smallest ladder climb to bricks.

20.3 C++ Code

```
class Solution {
public:
    int furthestBuilding(vector<int>& heights, int bricks, int ladders) {
        priority_queue<int, vector<int>, greater<int>> ladderClimbs;

        for (int i = 0; i + 1 < heights.size(); i++) {
            int climb = heights[i + 1] - heights[i];

            if (climb <= 0) {
                continue;
            }

            ladderClimbs.push(climb);

            if (ladderClimbs.size() > ladders) {
                bricks -= ladderClimbs.top();
                ladderClimbs.pop();
            }

            if (bricks < 0) {
                return i;
            }
        }

        return heights.size() - 1;
    }
};
```

```

    }
}

return heights.size() - 1;
}
};

```

20.4 Intuition

The heap stores climbs currently assigned to ladders.
 If too many climbs use ladders, the smallest ladder climb should instead use bricks.
 This keeps ladders reserved for the largest climbs.

20.5 Complexity

Time:

$O(n \log \text{ladders})$

Space:

$O(\text{ladders})$

21. Full Problem Walkthrough 11 — LeetCode 502: IPO

21.1 Problem Summary

You have initial capital w .

Each project has:

- capital requirement
- profit

You can do at most k projects.

After completing a project, profit is added to capital.

Return maximum final capital.

21.2 How to Identify Heap

At any time:

- only projects with capital requirement $\leq$ current capital are available
- among available projects, choose highest profit

Use:

7. sort projects by capital
 8. max heap of available profits
-

21.3 C++ Code

```

class Solution {
public:
    int findMaximizedCapital(int k, int w, vector<int>& profits, vector<int>& capital)

```

```

{
    vector<pair<int, int>> projects;

    for (int i = 0; i < profits.size(); i++) {
        projects.push_back({capital[i], profits[i]});
    }

    sort(projects.begin(), projects.end());

    priority_queue<int> availableProfits;

    int i = 0;
    int n = projects.size();

    while (k > 0) {
        while (i < n && projects[i].first <= w) {
            availableProfits.push(projects[i].second);
            i++;
        }

        if (availableProfits.empty()) {
            break;
        }

        w += availableProfits.top();
        availableProfits.pop();

        k--;
    }

    return w;
}
};

```

21.4 Complexity

Sorting:

$O(n \log n)$

Heap operations:

$O(n \log n)$

Total:

$O(n \log n)$

Space:

$O(n)$

22. Full Problem Walkthrough 12 — LeetCode 767: Reorganize String

22.1 Problem Summary

Rearrange string so no two adjacent characters are equal.

If impossible, return empty string.

Example:

`s = "aab"`

Output:

`"aba"`

22.2 How to Identify Heap

Need repeatedly choose most frequent remaining character, but avoid same as previous.

Use max heap by frequency.

22.3 C++ Code

```
class Solution {
public:
    string reorganizeString(string s) {
        vector<int> freq(26, 0);

        for (char c : s) {
            freq[c - 'a']++;
        }

        priority_queue<pair<int, char>> pq;

        for (int i = 0; i < 26; i++) {
            if (freq[i] > 0) {
                pq.push({freq[i], char('a' + i)});
            }
        }

        string result;

        while (pq.size() >= 2) {
```

```

    auto first = pq.top();
    pq.pop();

    auto second = pq.top();
    pq.pop();

    result += first.second;
    result += second.second;

    first.first--;
    second.first--;

    if (first.first > 0) {
        pq.push(first);
    }

    if (second.first > 0) {
        pq.push(second);
    }
}

if (!pq.empty()) {
    if (pq.top().first > 1) {
        return "";
    }

    result += pq.top().second;
}

return result;
}
};

```

22.4 Why Taking Top Two Works

At each step, choosing two most frequent different characters reduces the largest counts while avoiding adjacent duplicates.

If one character remains with count > 1, impossible.

22.5 Complexity

Time:

$O(n \log 26)$

Effectively $O(n)$.

Space:

$O(1)$

for lowercase letters.

23. Full Problem Walkthrough 13 — LeetCode 621: Task Scheduler

23.1 Problem Summary

Tasks are uppercase letters.

Same task must be separated by at least n intervals.

Return least intervals needed.

Example:

tasks = {'A','A','A','B','B','B'}

$n = 2$

Output:

8

Schedule:

A B idle A B idle A B

23.2 Heap Simulation Approach

Use max heap for task counts.

Each cycle has length $n + 1$.

In each cycle, schedule up to $n + 1$ most frequent different tasks.

23.3 C++ Code

```
class Solution {
public:
    int leastInterval(vector<char>& tasks, int n) {
        vector<int> freq(26, 0);

        for (char task : tasks) {
            freq[task - 'A']++;
        }

        priority_queue<int> pq;

        for (int count : freq) {
            if (count > 0) {
                pq.push(count);
            }
        }
    }
};
```

```

    }
}

int time = 0;

while (!pq.empty()) {
    vector<int> used;
    int cycle = n + 1;

    for (int i = 0; i < cycle; i++) {
        if (!pq.empty()) {
            int count = pq.top();
            pq.pop();

            count--;

            if (count > 0) {
                used.push_back(count);
            }
        }

        time++;

        if (pq.empty() && used.empty()) {
            break;
        }
    }

    for (int count : used) {
        pq.push(count);
    }
}

return time;
}
};

```

23.4 Complexity

Time:

$O(T \log 26)$

Effectively $O(T)$, where T is number of tasks plus idle simulation.

Space:

$O(1)$

23.5 Alternative Formula

There is a math solution:

```
max(tasks.size(), (maxFreq - 1) * (n + 1) + numberOfMaxFreqTasks)
```

Heap simulation is easier to understand.

24. Full Problem Walkthrough 14 — LeetCode 632: Smallest Range Covering Elements from K Lists

24.1 Problem Summary

Given k sorted lists, find smallest range $[l, r]$ that includes at least one number from each list.

24.2 How to Identify Heap

Need maintain one current element from each list.

Range is:

```
currentMax - currentMin
```

Use min heap to advance the list containing current minimum.

Track current maximum separately.

24.3 C++ Code

```
class Solution {
public:
    vector<int> smallestRange(vector<vector<int>>& nums) {
        using T = tuple<int, int, int>; // value, list index, element index

        priority_queue<T, vector<T>, greater<T>> pq;

        int currentMax = INT_MIN;

        for (int i = 0; i < nums.size(); i++) {
            pq.push({nums[i][0], i, 0});
            currentMax = max(currentMax, nums[i][0]);
        }

        int bestLeft = 0;
        int bestRight = INT_MAX;
```

```

while (pq.size() == nums.size()) {
    auto [value, listIndex, elementIndex] = pq.top();
    pq.pop();

    if (currentMax - value < bestRight - bestLeft) {
        bestLeft = value;
        bestRight = currentMax;
    }

    if (elementIndex + 1 == nums[listIndex].size()) {
        break;
    }

    int nextValue = nums[listIndex][elementIndex + 1];
    pq.push({nextValue, listIndex, elementIndex + 1});
    currentMax = max(currentMax, nextValue);
}

return {bestLeft, bestRight};
}
};

```

24.4 Why Move the Minimum?

The current range is determined by min and max.

To potentially shrink the range, we must increase the minimum.

Moving any other list would not improve the left boundary.

24.5 Complexity

Let:

- N = total number of elements
- k = number of lists

Time:

$O(N \log k)$

Space:

$O(k)$

25. Mini Problem 1 — Kth Smallest in Sorted Matrix Using Heap

Alternative to binary search on answer.

```

class Solution {
public:
    int kthSmallest(vector<vector<int>>& matrix, int k) {
        int n = matrix.size();

        using T = tuple<int, int, int>; // value, row, col
        priority_queue<T, vector<T>, greater<T>> pq;

        for (int r = 0; r < n; r++) {
            pq.push({matrix[r][0], r, 0});
        }

        int value = 0;

        while (k--) {
            auto [v, r, c] = pq.top();
            pq.pop();

            value = v;

            if (c + 1 < n) {
                pq.push({matrix[r][c + 1], r, c + 1});
            }
        }

        return value;
    }
};

```

Time:

$O(k \log n)$

26. Mini Problem 2 — Connect Sticks

Classic heap problem.

Given stick lengths, repeatedly connect two smallest sticks.

Cost is sum.

Return minimum total cost.

```

int connectSticks(vector<int>& sticks) {
    priority_queue<int, vector<int>, greater<int>> pq(sticks.begin(), sticks.end());
}

```

```
int cost = 0;

while (pq.size() > 1) {
    int a = pq.top();
    pq.pop();

    int b = pq.top();
    pq.pop();

    cost += a + b;
    pq.push(a + b);
}

return cost;
}
```

Why min heap?

Always combining two smallest minimizes total cost.

This is Huffman coding logic.

27. Mini Problem 3 — Relative Ranks

Sort is simpler, but heap also works.

Need assign ranks by score descending.

Max heap of {score, index}:

```
priority_queue<pair<int, int>> pq;

for (int i = 0; i < score.size(); i++) {
    pq.push({score[i], i});
}
```

Pop in rank order.

28. Mini Problem 4 — Distant Barcodes

Same pattern as Reorganize String.

Use max heap by frequency and repeatedly place two most frequent different values.

29. Advanced Concept — Lazy Deletion

C++ priority queue cannot remove arbitrary elements.

For sliding window median, old elements leave the window but may be buried inside heap.

Lazy deletion means:

9. Mark old value as invalid in hash map.
10. When that value reaches heap top, pop it.
11. Repeat until heap top is valid.

Basic pruning:

```
void prune(priority_queue<int>& heap, unordered_map<int, int>& delayed) {
    while (!heap.empty()) {
        int x = heap.top();

        if (!delayed.count(x)) {
            break;
        }

        delayed[x]--;

        if (delayed[x] == 0) {
            delayed.erase(x);
        }

        heap.pop();
    }
}
```

For two heaps and duplicates, implementation is more complex.

This is used in:

- Sliding Window Median
- delayed removal priority problems

30. Common Edge Cases for Heap Problems

30.1 Empty Heap

Never call:

```
pq.top()
```

if heap may be empty.

Check:

```
if (!pq.empty())
```

30.2 k Greater Than Unique Count

Some problems guarantee this will not happen.
If not, handle carefully.

30.3 k = 1

Top k / kth largest often becomes max/min.
Heap still works.

30.4 Duplicate Values

Heap handles duplicates.
But lazy deletion with duplicates requires counts.

30.5 Custom Comparator Direction

For min heap:

```
return a.value > b.value;
```

This feels reversed.
Be careful.

30.6 Integer Overflow

Distances:

```
1LL * x * x + 1LL * y * y
```

Sums/costs may need long long.

31. Common Mistakes and Wrong Code

Mistake 1 — Using Max Heap for Kth Largest Stream

Wrong:

```
priority_queue<int> pq;
```

This gives largest, not kth largest.
Use min heap of size k.

Mistake 2 — Popping Before Checking Size

Correct:

```
pq.push(x);
```

```
if (pq.size() > k) {
```

```
    pq.pop();  
}
```

Mistake 3 — Thinking Heap Is Sorted

A heap only guarantees top priority.
Internal order is not sorted.

Mistake 4 — Wrong Comparator for Min Heap

Wrong:

```
return a.val < b.val;
```

for min heap custom comparator.
Correct often:

```
return a.val > b.val;
```

Mistake 5 — Forgetting to Push Next Element in K-Way Merge

After popping from one list, push the next node/value from that same list.

Mistake 6 — Using Heap When Sorting Is Required

If output must be sorted, sorting may be clearer.

32. Interview Explanation Templates

32.1 Kth Largest Stream

I maintain a min heap of size k containing the k largest values seen so far. The smallest among these k values is the k th largest, so it stays at the heap top.

32.2 Top K

I use a heap of size k . For each candidate, I push it into the heap. If heap size exceeds k , I remove the lowest-priority candidate. At the end, the heap contains the top k items.

32.3 K-Way Merge

Since each list is sorted, the next global smallest element must be among the current heads of the lists. I store those heads in a min heap and repeatedly pop the smallest, then push the next element from that same list.

32.4 Median Stream

I keep two heaps: a max heap for the lower half and a min heap for the upper half. Balancing their sizes lets me get the median in $O(1)$, while insertion costs $O(\log n)$.

32.5 Meeting Rooms

I sort meetings by start time and keep a min heap of end times for active meetings. If the earliest ending meeting is done before the current starts, I reuse that room. Otherwise, I need another room.

33. Complexity Patterns

Kth largest stream	$O(\log k)$ per add	$O(k)$
Kth largest array	$O(n \log k)$	$O(k)$
Top K frequent	$O(n + u \log k)$	$O(u + k)$
K closest points	$O(n \log k)$	$O(k)$
Last Stone Weight	$O(n \log n)$	$O(n)$
Merge k lists	$O(N \log k)$	$O(k)$
Median stream	$O(\log n)$ add, $O(1)$ median	$O(n)$
Meeting Rooms II	$O(n \log n)$	$O(n)$
Furthest Building	$O(n \log \text{ladders})$	$O(\text{ladders})$
IPO	$O(n \log n)$	$O(n)$

34. Time-Boxed Practice Guidance

Easy Heap Problems

Target:

10–20 minutes

Examples:

- Kth Largest Element in a Stream
 - Last Stone Weight
 - Kth Largest Element in an Array, heap version
-

Medium Heap Problems

Target:

20–40 minutes

Examples:

- Top K Frequent Elements
- K Closest Points
- Meeting Rooms II
- Furthest Building
- Reorganize String

Hard Heap Problems

Target:

40–70 minutes

Examples:

- Merge k Sorted Lists
 - Find Median from Data Stream
 - IPO
 - Smallest Range Covering Elements from K Lists
 - Sliding Window Median
-

35. Expanded Practice Set

35.1 Must-Do Easy

- 1.1. Kth Largest Element in a Stream
 - 2.2. Last Stone Weight
 - 3.3. Kth Largest Element in an Array, heap version
 - 4.4. Relative Ranks
-

35.2 Must-Do Medium

- 5.5. Top K Frequent Elements
 - 6.6. Top K Frequent Words
 - 7.7. K Closest Points to Origin
 - 8.8. Meeting Rooms II
 - 9.9. Kth Smallest Element in a Sorted Matrix
 - 10.10. Furthest Building You Can Reach
 - 11.11. Reorganize String
 - 12.12. Distant Barcodes
 - 13.13. Task Scheduler
 - 14.14. Sort Characters By Frequency
-

35.3 Strong Interview Practice

- 15.15. Merge k Sorted Lists
 - 16.16. Find Median from Data Stream
 - 17.17. IPO
 - 18.18. Smallest Range Covering Elements from K Lists
 - 19.19. Design Twitter
 - 20.20. Single-Threaded CPU
 - 21.21. Minimum Cost to Hire K Workers
 - 22.22. Task Scheduler
-

35.4 Hard / Stretch

- 23.23. Sliding Window Median
- 24.24. The Skyline Problem
- 25.25. Trapping Rain Water II
- 26.26. Smallest Range Covering Elements from K Lists
- 27.27. Employee Free Time

36. Suggested Study Order

Step 1 — Basic Priority Queue

- 12. Last Stone Weight
- 13. Kth Largest Element in a Stream
- 14. Kth Largest Element in an Array

Goal:

Learn max heap, min heap, size-k heap.

Step 2 — Top K

- 15. Top K Frequent Elements
- 16. K Closest Points
- 17. Top K Frequent Words

Goal:

Learn heap by custom priority.

Step 3 — K-Way Merge

- 18. Merge k Sorted Lists
- 19. Kth Smallest Element in a Sorted Matrix
- 20. Smallest Range Covering K Lists

Goal:

Learn min heap over multiple sorted sources.

Step 4 — Scheduling / Greedy

- 21. Meeting Rooms II
- 22. Task Scheduler
- 23. Reorganize String
- 24. Furthest Building
- 25. IPO

Goal:

Learn heap as greedy decision support.

Step 5 — Two Heaps

- 26. Find Median from Data Stream
- 27. Sliding Window Median

Goal:

Learn lower-half / upper-half balancing.

37. Pattern Decision Table

kth largest stream	min heap size k
kth largest array	min heap size k

Problem Clue	Heap Strategy
kth smallest	max heap size k
top k frequent	min heap by frequency
k closest	max heap by distance size k
repeatedly smash largest	max heap
merge k sorted lists	min heap of current heads
earliest ending resource	min heap of end times
median stream	two heaps
choose largest profit available	max heap
reserve ladders for largest climbs	min heap of ladder climbs
reorganize by frequency	max heap of counts

38. Final Cheat Sheet

Max Heap

```
priority_queue<int> pq;
```

Min Heap

```
priority_queue<int, vector<int>, greater<int>> > pq;
```

Kth Largest

```
priority_queue<int, vector<int>, greater<int>> > pq;

for (int x : nums) {
    pq.push(x);

    if (pq.size() > k) {
        pq.pop();
    }
}

return pq.top();
```

K Closest Points

```

priority_queue<pair<int, int>> pq;

for (int i = 0; i < points.size(); i++) {
    int d = dist(points[i]);

    pq.push({d, i});

    if (pq.size() > k) {
        pq.pop();
    }
}

```

Merge K Sorted Lists

```

struct Compare {
    bool operator()(ListNode* a, ListNode* b) {
        return a->val > b->val;
    }
};

priority_queue<ListNode*, vector<ListNode*>, Compare> pq;

```

Two Heaps Median

```

priority_queue<int> small;
priority_queue<int, vector<int>, greater<int>> large;

```

Meeting Rooms

```

priority_queue<int, vector<int>, greater<int>> endTimes;

```

Custom Min Heap

```

struct Compare {
    bool operator()(const vector<int>& a, const vector<int>& b) {
        return a[0] > b[0];
    }
};

```

39. What to Master Before Chapter 13

Before moving to Stack, make sure you can:

- Use max heap in C++
 - Use min heap in C++
 - Know top, push, pop
 - Keep heap size k
 - Solve kth largest stream
 - Solve top k frequent elements
 - Solve k closest points
 - Merge k sorted lists
 - Use heap for meeting rooms
 - Use two heaps for median stream
 - Write custom heap comparator
 - Understand why heap is not fully sorted
 - Know when heap beats sorting
 - Know when Quickselect beats heap
 - Handle empty heap safely
-

40. Core Takeaway

A heap is for repeatedly getting the current best item.

The core question is:

Do I repeatedly need the smallest, largest, or highest-priority element?

If yes, heap is likely useful.

Common heap patterns:

min heap size k	-> kth largest / top k largest
max heap size k	-> kth smallest / top k smallest
min heap of heads	-> merge k sorted sources
min heap of end times	-> scheduling rooms/resources
two heaps	-> streaming median
max heap by frequency	-> rearrangement / task selection

Heap does not fully sort.

It only guarantees that the top element is the current best.

That is exactly what makes it efficient for priority-based problems.